

Multi-Level Paging and the TLB

2026 Semester 2 COMPSCI 340: Operating Systems

Talia Xu

Lecture 10

2.1.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

This Lecture Makes Page Translation Practical

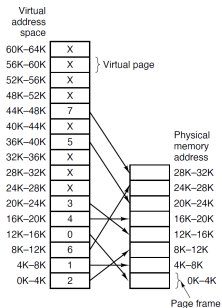
Last lecture split a virtual address into a virtual page number and an offset, then used a page table to find the physical page number.

Today we solve two problems with that basic design:

one large page table → multi-level page tables → several lookups per access → the TLB

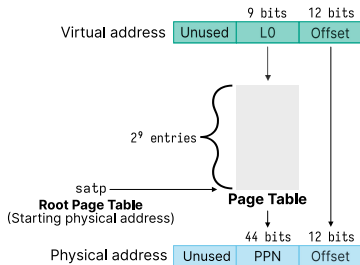
Multi-level tables reduce the space used by page tables. The TLB reduces the time spent walking them.

A Flat Page Table Reserves an Entry for Every Virtual Page



A process normally uses only a few regions of its virtual address space. A flat table still contains PTEs for the unused gaps.

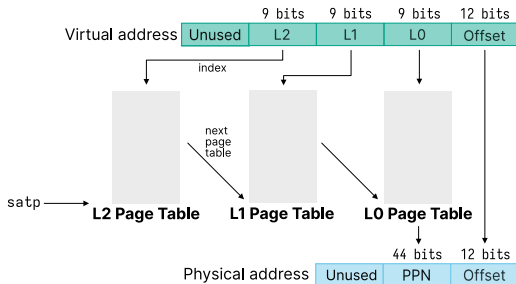
A Page-Sized Table Holds 512 Entries



$$\frac{4096 \text{ bytes per page}}{8 \text{ bytes per PTE}} = 512 = 2^9 \text{ PTEs}$$

A page-sized table can therefore use 9 virtual-address bits as its index. More address bits require more levels.

Multi-Level Page Tables Allocate Lower Tables Only Where Needed



An entry in the top-level table either points to a lower-level table or says that the whole covered region is unmapped.

No lower-level table is needed for an unused region.

Each Level Uses Its Own Part of the Virtual Address

A 30-bit virtual address with 4 KiB pages can be divided as:

9-bit L1 index	9-bit L0 index	12-bit offset
choose L1 entry	choose L0 entry	choose byte in page

The L1 entry points to an L0 table. The L0 entry supplies the physical page number. The 12-bit offset is copied unchanged.

Compare One-Level and Two-Level Tables

Consider a 30-bit virtual address, 4 KiB pages, and 8-byte PTEs. The process currently uses only the page containing 0x3FFFF008.

One-level page table

Two-level page table

Sparse Address Spaces Need Far Less Page-Table Memory

The 30-bit address has 18 VPN bits.

$$\text{flat table} = 2^{18} \times 8 \text{ bytes} = 2 \text{ MiB}$$

With only one mapped page, the two-level design needs:

$$\text{one 4 KiB L1 table} + \text{one 4 KiB L0 table} = 8 \text{ KiB}$$

The saving comes from the unused L1 entries having no L0 tables below them. A densely used address space needs more of those lower tables.

Walk One Address Through Two Levels

Use virtual address 0x3FFFF008 and the format:

11111111_11111111_000000001000

The selected L0 PTE contains physical page number 0x2A.

The Walk Replaces the VPN but Preserves the Offset

1. L1 index 111111111 selects entry 511 in the root table.
2. That entry points to an L0 table.
3. L0 index 111111111 selects entry 511 in that table.
4. Its PTE supplies PPN 0x2A.
5. Append offset 0x008: physical address 0x2A008.

The indices guide the page-table walk; only the final PTE contributes the physical page number.

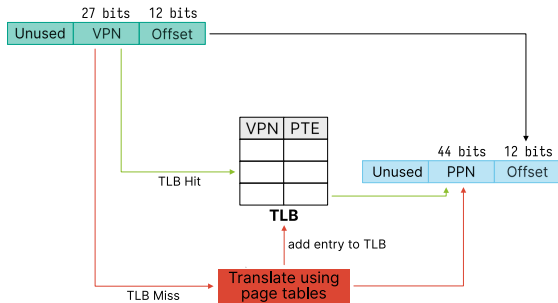
Saving Space Makes an Uncached Translation Slower

A two-level walk must read:

1. the L1 PTE,
2. the L0 PTE, and
3. the requested instruction or data.

Programs repeatedly access the same small set of pages. We can cache the recently used VPN $\rightarrow$ PPN results instead of repeating the full walk every time.

The TLB Caches Recent Page-Table Results



The Translation Lookaside Buffer (TLB) is a small hardware cache of recent VPN → PPN translations.

A TLB Hit Skips the Page-Table Walk

TLB hit	TLB miss
Find VPN in TLB	VPN is not in TLB
Obtain PPN	Walk L1, then L0
Access the requested byte	Insert VPN → PPN into TLB
	Access the requested byte

TLB miss: the translation is not cached, so the CPU walks the page table. The page may already be in RAM.

Page fault: the PTE says the page is not currently in RAM, so the CPU traps to the kernel. The kernel must make the page resident before the access can continue.

Estimate the Effective Access Time

Assume:

- a two-level page table,
- an 80% TLB hit rate,
- a 10 ns TLB lookup, and
- 100 ns for each memory access.

A hit needs the requested memory access. A miss needs two page-table accesses followed by the requested memory access.

Most Accesses Avoid the Two-Level Walk

$$\text{Hit} = 10 + 100 = 110 \text{ ns}$$

$$\text{Miss} = 10 + 2(100) + 100 = 310 \text{ ns}$$

$$\text{EAT} = 0.8(110) + 0.2(310) = 150 \text{ ns}$$

Without the TLB, every access would need three memory accesses, or 300 ns in this simplified model.

A Context Switch Must Keep TLB Entries Separate

The same VPN can map to different physical pages in different processes. A cached translation must therefore be associated with the correct address space.

Two common approaches are:

- invalidate relevant TLB entries during a context switch, or
- tag entries with an address-space identifier (ASID).

ASIDs allow translations from several processes to remain cached safely.

Multi-Level Tables Save Space; the TLB Saves Time

- A flat page table allocates a PTE for every possible virtual page.
- A multi-level table creates lower tables only for mapped regions.
- Each address field selects one level; the offset is unchanged.
- A TLB hit reuses a recent translation and skips the page-table walk.
- A TLB miss walks the tables; it is different from a page fault.

Next: when a valid page is not in RAM, the kernel may need to choose which resident page to replace.